

Labo 03 — Labo pratique : vie, signaux et mort d'un conteneur

Objectif : provoquer vous-même chaque comportement du cours — l'arrêt immédiat, les 10 secondes d'agonie, le code 137, le redémarrage automatique.

Prérequis — Labos 01 et 02 terminés. Images `alpine` et `nginx:alpine` présentes.

Fichiers fournis — `files/demarrage-casse.sh` et `files/demarrage-correct.sh`, utilisés à l'étape 3.

Étape 1 — Les états, un par un

```
docker create --name etat alpine sleep 120
docker ps -a --filter name=etat --format 'table {{.Names}}\t{{.Status}}'
```

Observez le statut `Created` : le conteneur existe, aucun processus ne tourne.

```
docker start etat
docker ps --filter name=etat --format '{{.Status}}'
docker pause etat && docker ps --filter name=etat --format '{{.Status}}'
docker unpause etat && docker ps --filter name=etat --format '{{.Status}}'
docker stop etat && docker ps -a --filter name=etat --format '{{.Status}}'
docker start etat && docker ps --filter name=etat --format '{{.Status}}'
docker rm -f etat
```

Observez la succession `Up 1 second`, `Up 3 seconds (Paused)`, `Up 5 seconds`, `Exited (137)`, puis de nouveau `Up`.

Explication. Un conteneur `Exited` est redémarrable : il a gardé sa configuration et sa couche d'écriture. `docker run` n'est rien d'autre que `create` + `start`.

Étape 2 — Pourquoi un conteneur s'arrête tout seul

```
docker run --name essai1 alpine
docker ps -a --filter name=essai1 --format '{{.Status}}'
```

Observez `Exited (0)` : la commande par défaut d'`alpine` est `/bin/sh`, qui, sans entrée, se termine immédiatement.

```
docker run -d --name essai2 nginx:alpine
docker ps --filter name=essai2 --format '{{.Status}}'
```

Observez `Up` : nginx tourne au premier plan.

```
docker run --rm alpine sh -c 'sleep 60 & echo "lance en arriere-plan"'
```

Observez que la commande revient **immédiatement**, alors qu'un `sleep 60` a bien été lancé.

Explication. Le `&` a détaché `sleep` ; le shell a exécuté `echo` puis s'est terminé. Le PID 1 étant mort, le conteneur est détruit, `sleep` avec lui. C'est le piège numéro un des scripts de démarrage.

```
docker rm essai1 ; docker rm -f essai2
```

Étape 3 — Le script de démarrage cassé, et sa correction

Copiez les deux scripts fournis :

```
mkdir -p ~/labo-docker/03 && cd ~/labo-docker/03
cp <chemin-du-labo>/files/*.sh . && chmod +x *.sh
cat demarrage-casse.sh demarrage-correct.sh
```

Exécutez le premier **dans** un conteneur, en montant le dossier courant :

```
docker run --rm -v "$PWD":/scripts alpine /scripts/demarrage-casse.sh
docker ps -a --latest --format '{{.Status}}'
```

Observez le message affiché, puis un retour immédiat au prompt.

```
docker run -d --name correct -v "$PWD":/scripts alpine /scripts/demarrage-correct.sh
docker ps --filter name=correct --format '{{.Status}}'
docker top correct
```

Observez que le conteneur reste `Up`, et que `docker top` montre `sleep` — et **pas** de processus `sh` parent.

Explication. Le `exec` du second script a **remplacé** le shell par la commande finale, qui hérite du PID 1. C'est exactement ce que fait la forme `exec` d'un `ENTRYPOINT`, vue au labo 04.

```
docker rm -f correct
```

Étape 4 — Les 10 secondes d'agonie

Mesurez un arrêt sur un processus qui ignore `SIGTERM` :

```
docker run -d --name veille alpine sleep 300
time docker stop veille
docker inspect --format 'code={{.State.ExitCode}} oom={{.State.OOMKilled}}' veille
docker rm veille
```

Observez `real 0m10.1s` et `code=137 oom=false`.

Recommencez avec un mini-init :

```
docker run -d --init --name veille alpine sleep 300
time docker stop veille
docker inspect --format 'code={{.State.ExitCode}}' veille
docker rm veille
```

Observez `real 0m0.1s` et `code=143`.

Et avec une application qui gère correctement ses signaux :

```
docker run -d --name web nginx:alpine
time docker stop web
docker inspect --format 'code={{.State.ExitCode}}' web
docker rm web
```

Observez un arrêt instantané et `code=0`.

Explication. Trois comportements, trois causes. `sleep` en PID 1 **ignore** `SIGTERM` (protection du noyau) : Docker attend puis tue → `137`. Avec `--init`, `sleep` n'est plus PID 1, l'action par défaut s'applique → `143`. nginx installe un gestionnaire de signal et se termine proprement → `0`. Ces dix secondes multipliées par vos conteneurs, c'est la durée inexpliquée de vos redéploiements.

Vous pouvez raccourcir le délai de grâce — sans corriger la cause :

```
docker run -d --name veille alpine sleep 300
time docker stop -t 2 veille
docker rm veille
```

Observez `real 0m2.1s`, toujours avec le code `137`.

Étape 5 — Lire les codes de sortie

```
docker run --rm alpine sh -c 'exit 0' ; echo "code=$?"
docker run --rm alpine sh -c 'exit 3' ; echo "code=$?"
docker run --rm alpine commande-absente ; echo "code=$?"
```

Observez `0`, `3`, puis `127` accompagné d'un message `exec: "commande-absente": executable file not found in $PATH`.

```
docker run -d --name tue alpine sleep 300
docker kill tue
docker inspect --format 'code={{.State.ExitCode}}' tue
docker rm tue
```

Observez `137`, immédiatement cette fois : `kill` n'attend pas.

Explication. Au-delà de 128, le code indique une mort par signal : `code - 128` donne le numéro du signal. `127` en revanche est une erreur de lancement : l'application n'a jamais démarré.

Étape 6 — `exec` contre `attach`

```
docker run -d --name web nginx:alpine
docker exec web nginx -v
docker exec -it web sh
```

Dans le shell obtenu, tapez :

```
ps -o pid,comm
exit
```

Observez que `nginx` est le PID 1 et que votre `sh` porte un autre PID. En sortant du shell, le conteneur est **toujours** `Up`.

```
docker ps --filter name=web --format '{{.Status}}'
```

Explication. `exec` a créé un **nouveau** processus dans les namespaces du conteneur. Le quitter n'affecte pas le PID 1. `attach`, à l'inverse, vous brancherait sur nginx lui-même : un `Ctrl+C` l'arrêterait. Pour consulter les logs, utilisez toujours :

```
docker logs --tail 5 web
docker logs -f --since 1m web      # Ctrl+C ici n'arrête que l'affichage
```

Étape 7 — Les logs ne viennent que de `stdout`

```
docker run --rm --name logs-demo alpine sh -c \
  'echo "je vais sur stdout"; echo "je vais dans un fichier" > /tmp/app.log; sleep 1'
```

Observez que seule la première ligne apparaît.

```
docker run -d --name logs-demo alpine sh -c \
  'echo "visible"; echo "invisible" > /tmp/app.log; sleep 120'
docker logs logs-demo
docker exec logs-demo cat /tmp/app.log
docker rm -f logs-demo
```

Observez que `docker logs` affiche `visible` et que le contenu du fichier n'est accessible qu'en entrant dans le conteneur.

Explication. Le daemon ne capte que `stdout` et `stderr` du PID 1. C'est pourquoi une application conteneurisée doit logger sur la console — et pourquoi il ne faut pas configurer `logging.file.name` dans un Spring Boot conteneurisé.

Étape 8 — Redémarrage automatique

```
docker run -d --restart=on-failure:3 --name instable alpine \
  sh -c 'echo "demarrage $(date +%T)"; sleep 3; exit 1'
sleep 20
docker ps -a --filter name=instable --format '{{.Names}} {{.Status}}'
docker inspect --format 'redemarrages={{.RestartCount}} code={{.State.ExitCode}}' instable
docker logs instable
```

Observez un `RestartCount` de `3`, un statut `Exited (1)`, et **quatre** lignes « démarrage » dans les logs : la tentative initiale plus trois reprises.

Explication. Les logs s'accumulent d'une exécution à l'autre sur le même conteneur : la première ligne est la cause initiale. `.State`, en revanche, ne décrit que la **dernière** exécution.

```
docker rm instable
```

Vérifiez enfin l'incompatibilité annoncée en cours :

```
docker run --rm -d --restart=always nginx:alpine
```

Observez `docker: conflicting options: cannot specify both --restart and --rm.`

Étape 9 — Extraire une preuve d'un conteneur mort

```
docker run --name autopsie alpine sh -c 'echo "trace importante" > /rapport.txt; exit 2'
docker ps -a --filter name=autopsie --format '{{.Status}}'
docker cp autopsie:/rapport.txt ./rapport.txt
cat rapport.txt
```

Observez que le fichier est récupérable alors que le conteneur est `Exited (2)`.

```
docker rm autopsie
docker cp autopsie:/rapport.txt ./autre.txt
```

Observez l'erreur : le conteneur supprimé, tout est perdu.

Explication. Règle d'exploitation : **on inspecte avant de supprimer**. `docker cp`, `docker logs` et `docker inspect` fonctionnent sur un conteneur arrêté, jamais sur un conteneur supprimé.

Nettoyage

```
docker ps -a --filter ancestor=alpine --format 'table {{.Names}}\t{{.Status}}'
docker rm -f veille web correct instable autopsie essai essai2 tue etat 2>/dev/null
rm -f ~/labo-docker/03/rapport.txt
docker ps -a --filter ancestor=alpine --format '{{.Names}}'
```

Observez qu'aucun conteneur de ce labo ne subsiste. Les images `alpine` et `nginx:alpine` sont conservées.

Ce que vous devez pouvoir affirmer maintenant

- Un conteneur meurt avec son PID 1 — vous avez provoqué les trois cas.
- `sleep` en PID 1 ignore `SIGTERM` ; `--init` corrige le symptôme, `exec` la cause.
- `137` = tué, `143` = arrêté proprement, `127` = commande introuvable.
- `exec` crée un processus, `attach` se branche sur le PID 1.
- `docker logs` ne montre que `stdout` / `stderr`.
- `docker rm` détruit les logs et les preuves : on inspecte d'abord.